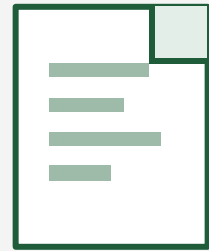

Software Architecture and Design

Oscar Chaparro

Too simple?

THE SUBMISSION



Challenge: a CLI tool that calls an API and pages through the data. One submission: ~30 lines of Python, the simplest way to meet the requirements. ✓ all requirements

THE SECOND REVIEWER

Proposed as the reference for a 10/10. The other reviewer wouldn't pass it to an interview — not enough sophisticated language features. **“Too simple.”**

REJECTED

Why simple wins: every part is a way to fail



API down → 1 way to fail



API down stale cache state out of sync retry storm

→ many

“Great software design looks simple because it eliminates as many failure modes as possible during the design stage.”

S. Goedecke, “Great software design looks underwhelming,” Mar 2025 · seangoedecke.com/great-software-design



Agenda

1 What architecture and design are

2 Design principles

3 From quality attributes to design decisions

4 Architectural styles

5 Designing inside an existing codebase

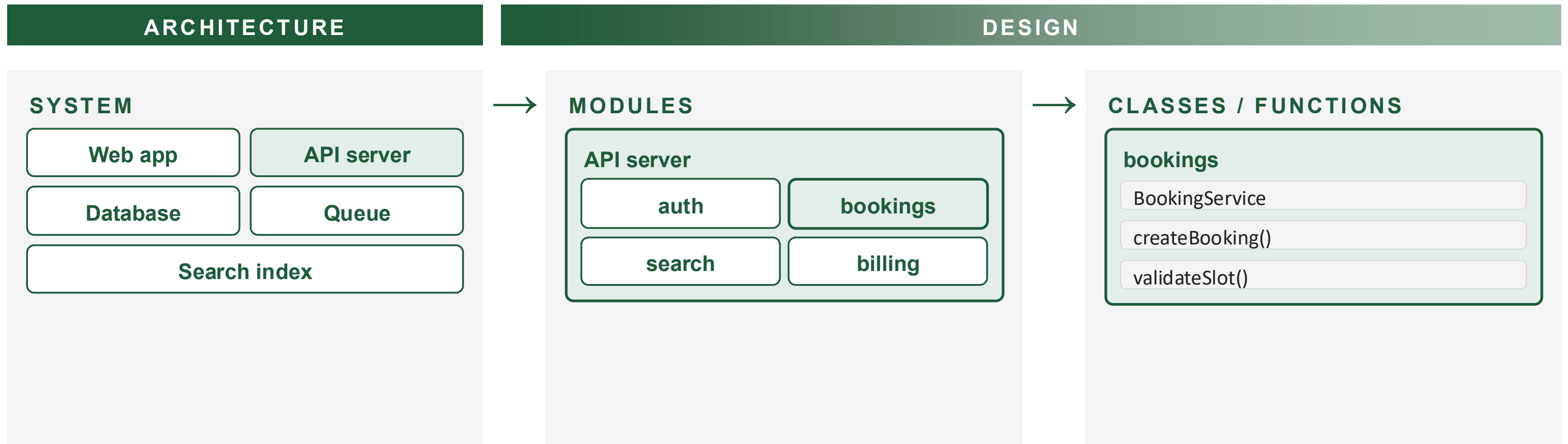
6 Recording design decisions

7 Agents and design

What architecture and design are

Structure, decisions, and reasons

Architecture and design



Architecture: the main parts of a system, how they connect, and the key decisions behind them — **and why**.

Design: the same questions at a smaller scale.

One system, several views

Module view

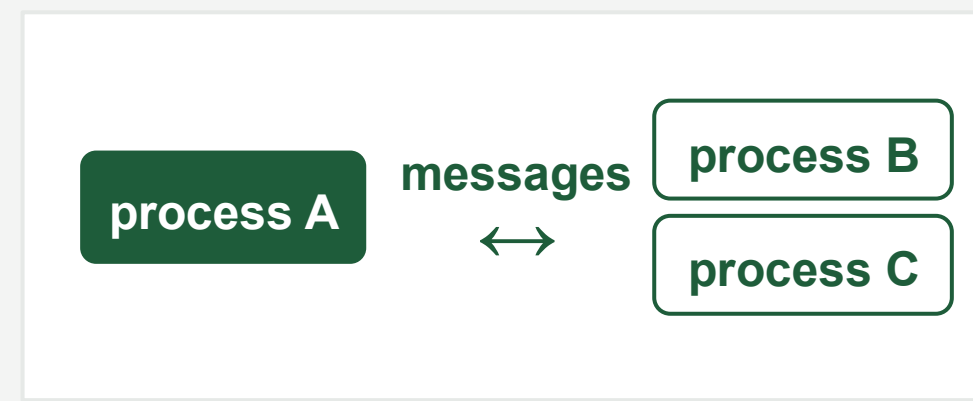


How is the code split, and what may depend on what? → **where do I make a change?**

Chromium layers

VS Code

Runtime view

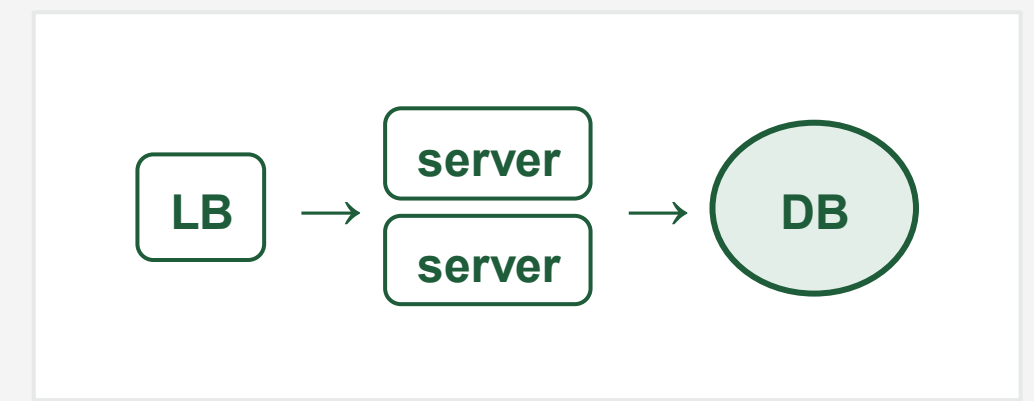


What runs, and how do the parts talk while it runs? → **what fails together?**

Chromium processes

VS Code extension host

Deployment view



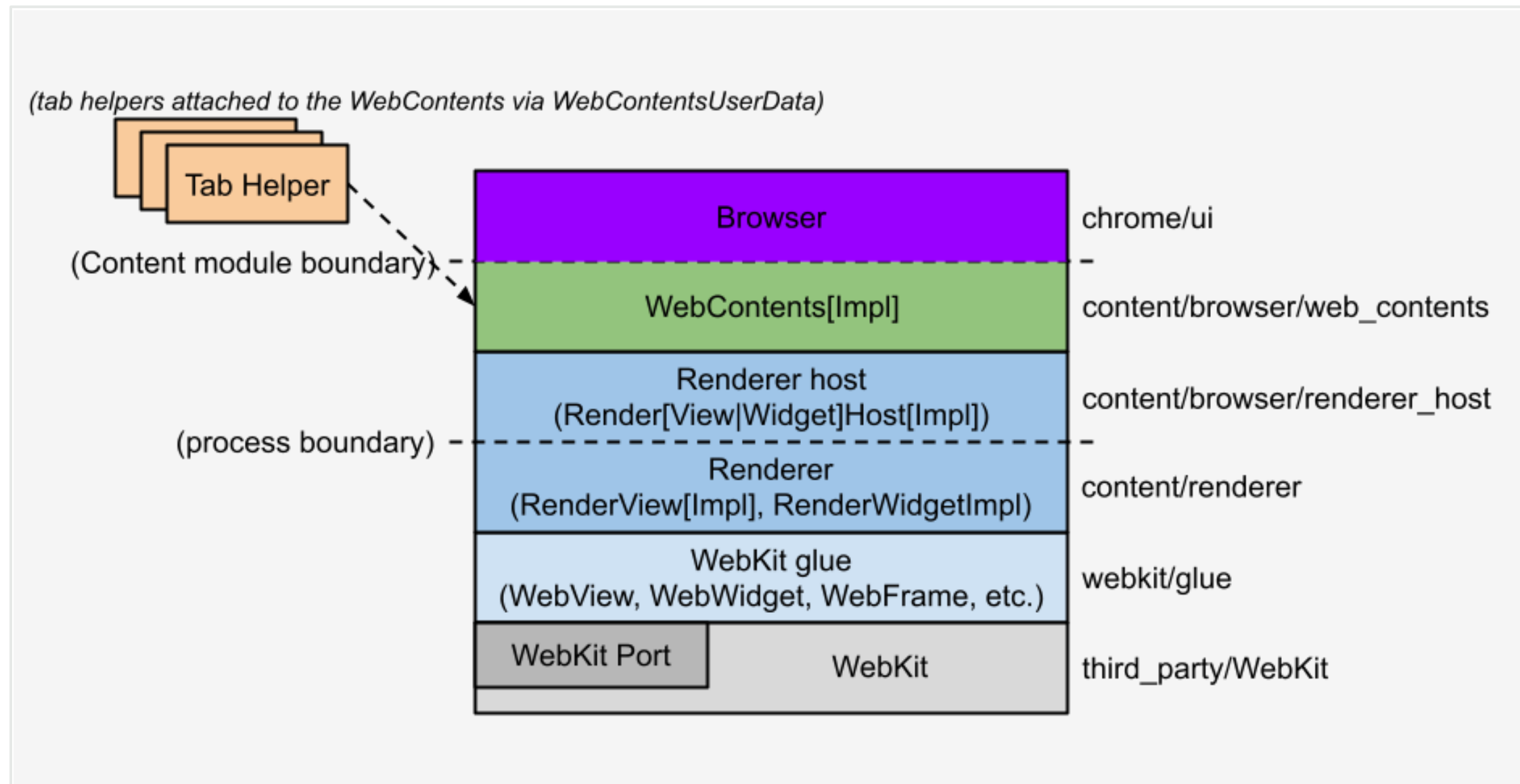
Which machines does it run on? → **where does it run, what does it cost?**

Stack Overflow

Different questions → different views. All of them are architecture.

Chromium: module view

MODULE VIEW



Source: chromium.org — “How Chromium Displays Web Pages” · CC BY 2.5

KEY COMPONENTS

WebKit (+ Port)

rendering engine; the Port connects it to OS services (loading, graphics)

Glue

converts WebKit types to Chromium types — the “WebKit embedding layer”

Renderer / Render host

carry commands and notifications across the process boundary

WebContents

reusable component that renders HTML into a view

Browser

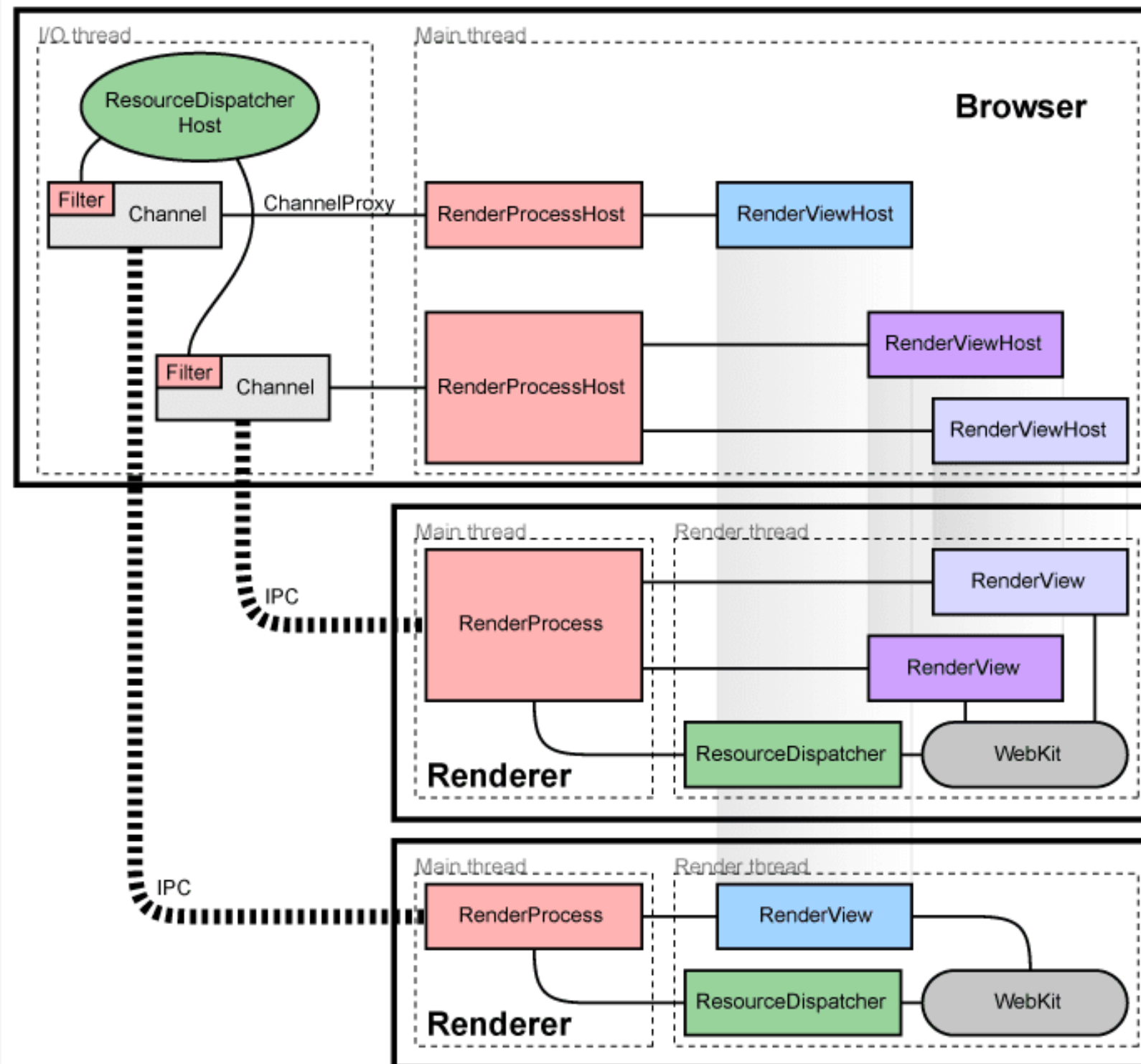
the browser window; holds many WebContents

Tab helpers

small add-ons attached to a WebContents (favicons, infobars)

Rule: no layer depends on a higher layer.

Chromium: runtime view



RUNTIME VIEW

KEY COMPONENTS

Browser process

one; runs the UI

Renderer process

one per site; runs WebKit in a sandbox

RenderProcessHost

browser-side handle on a renderer

Channel + IPC

async messages between processes

ResourceDispatcher (Host)

network requests go through the browser

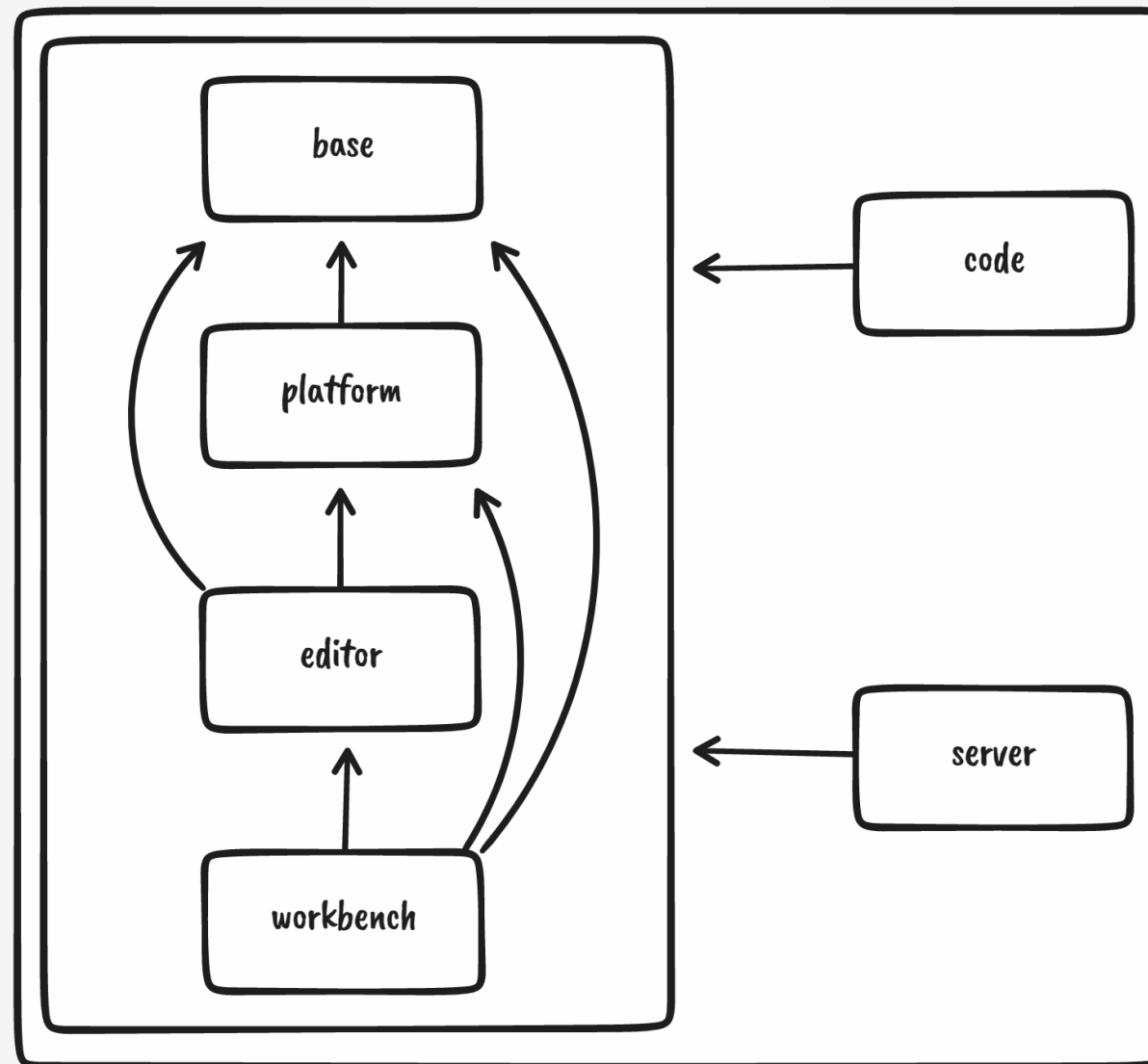
Why: a crash shows a “sad tab”, not a dead browser. **Cost:** more memory.

IPC = inter-process communication: messages between processes, which don't share memory.

Source: chromium.org — “Multi-process Architecture” design document · CC BY 2.5

VS Code: module view

MODULE VIEW



KEY COMPONENTS

base

general utilities and UI building blocks

platform

service injection and shared services

editor

the Monaco editor core

workbench

the IDE UI: editors, panels, status bar

code / server

starts the app: Electron main process, windows, CLI / remote server

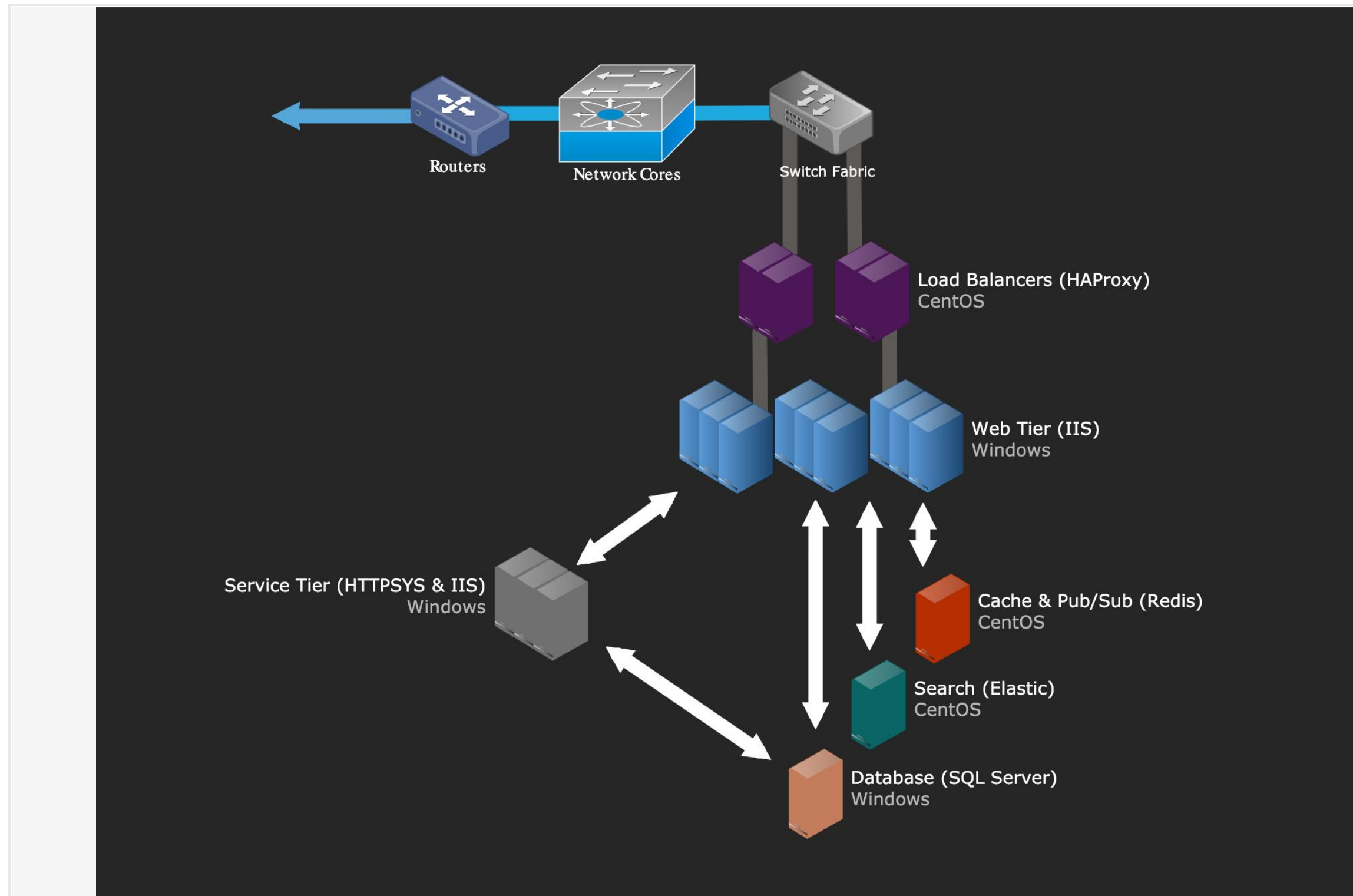
extension host

separate process; extensions use only the extension API

Source: github.com/microsoft/vscode/wiki — “Source Code Organization”

Rule: a layer uses only the layers below it. Extensions never touch the core directly.

Stack Overflow: deployment view



DEPLOYMENT VIEW

KEY COMPONENTS

Load balancers (HAProxy)

send each request to a web server

Web tier (IIS, ASP.NET MVC)

the main application that serves the sites

Service tier

separate servers for the tag engine and internal APIs — heavy work kept off the web tier

SQL Server

the main data store

Redis

shared cache and pub/sub between servers

Elasticsearch

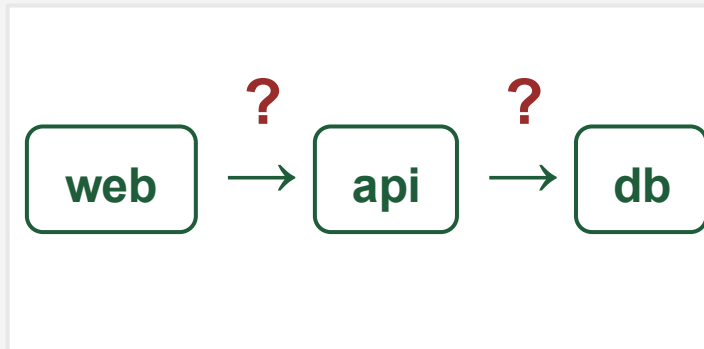
search

A huge site, mostly one application, on few servers — 2016 snapshot.

Source: N. Craver, "Stack Overflow: The Architecture – 2016 Edition," nickcraver.com

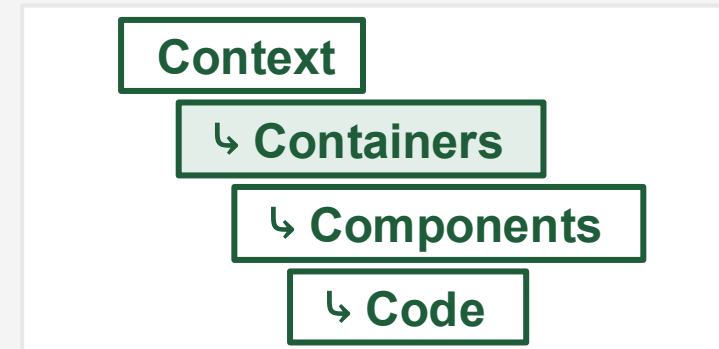
How people draw architectures

Boxes and arrows



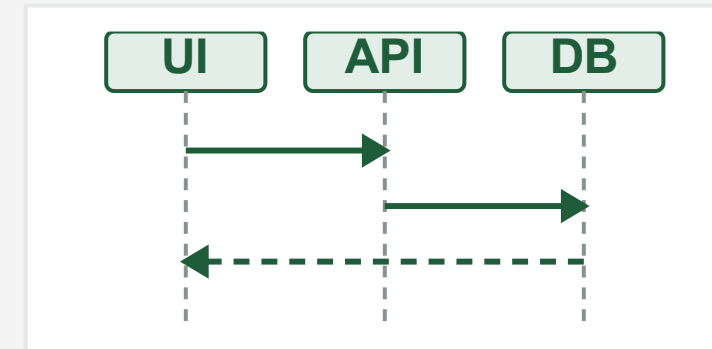
most common; fast, but what does an arrow mean?

C4



four zoom levels: context → containers → components → code

UML



mostly sequence, component and deployment diagrams

Diagrams as code

```
```mermaid
graph LR
 web --> api
 api --> db
```
```

Mermaid, PlantUML, Structurizr — kept in the repo

Whatever you use, add a legend: say what a box and an arrow mean.

C4 in brief

KEY FACTS

Simon Brown, c. 2011

4 zoom levels: context → containers → components → code

Fixed abstractions: person, system, container, component

Free notation: any shapes, plus a key and labels

Extra views: landscape, dynamic, deployment

POPULAR TOOLS

Diagrams as code

C4-PlantUML

Structurizr

Mermaid

Drawing tools

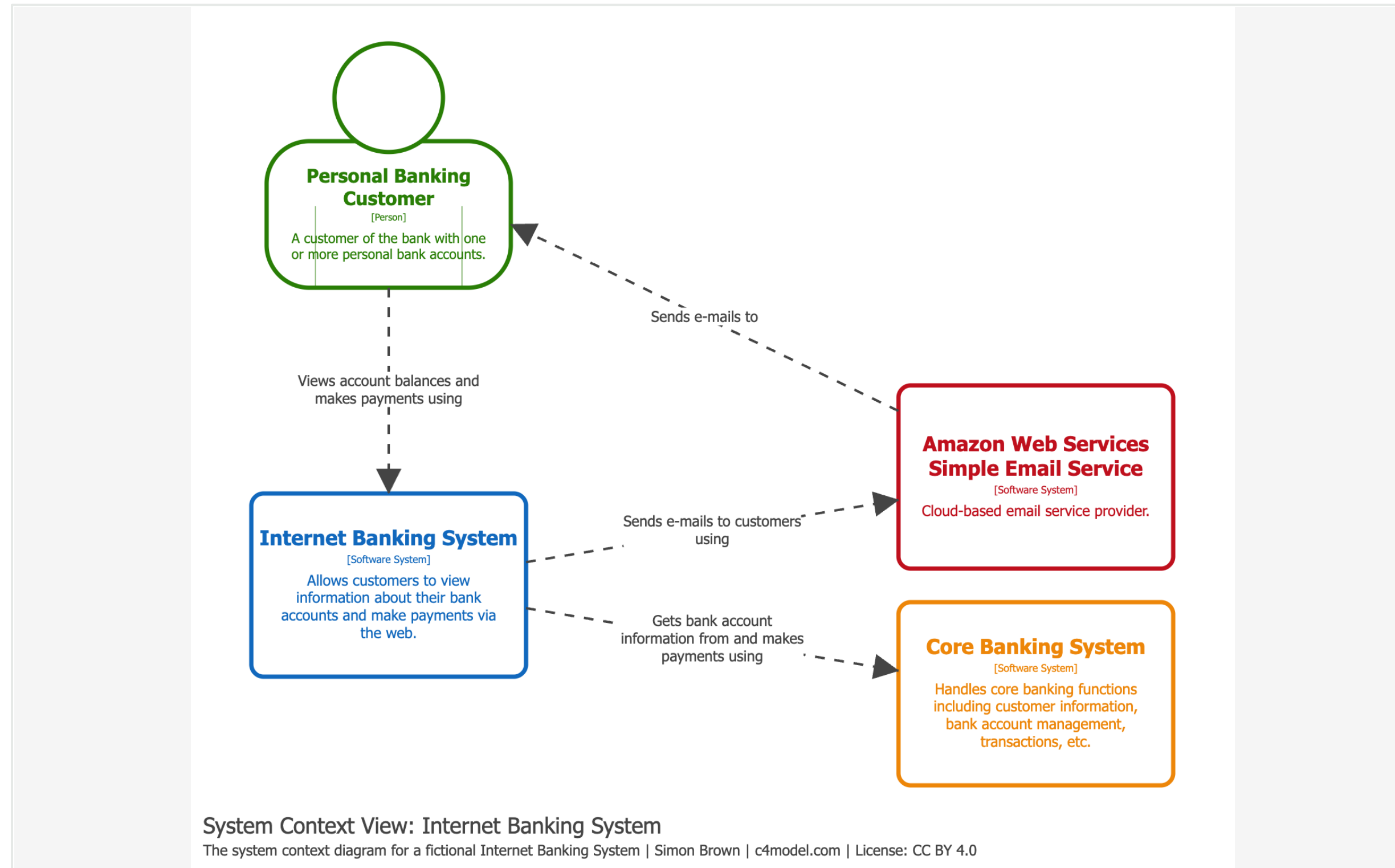
draw.io

IcePanel

Lucidchart / Miro

Source: c4model.com

C4: system context diagram



LEVEL 1 · CONTEXT

KEY COMPONENTS

Person

a user or role (the customer)

Software system (in scope)

the system we are building: Internet Banking

Existing systems

outside systems it talks to: mainframe, e-mail

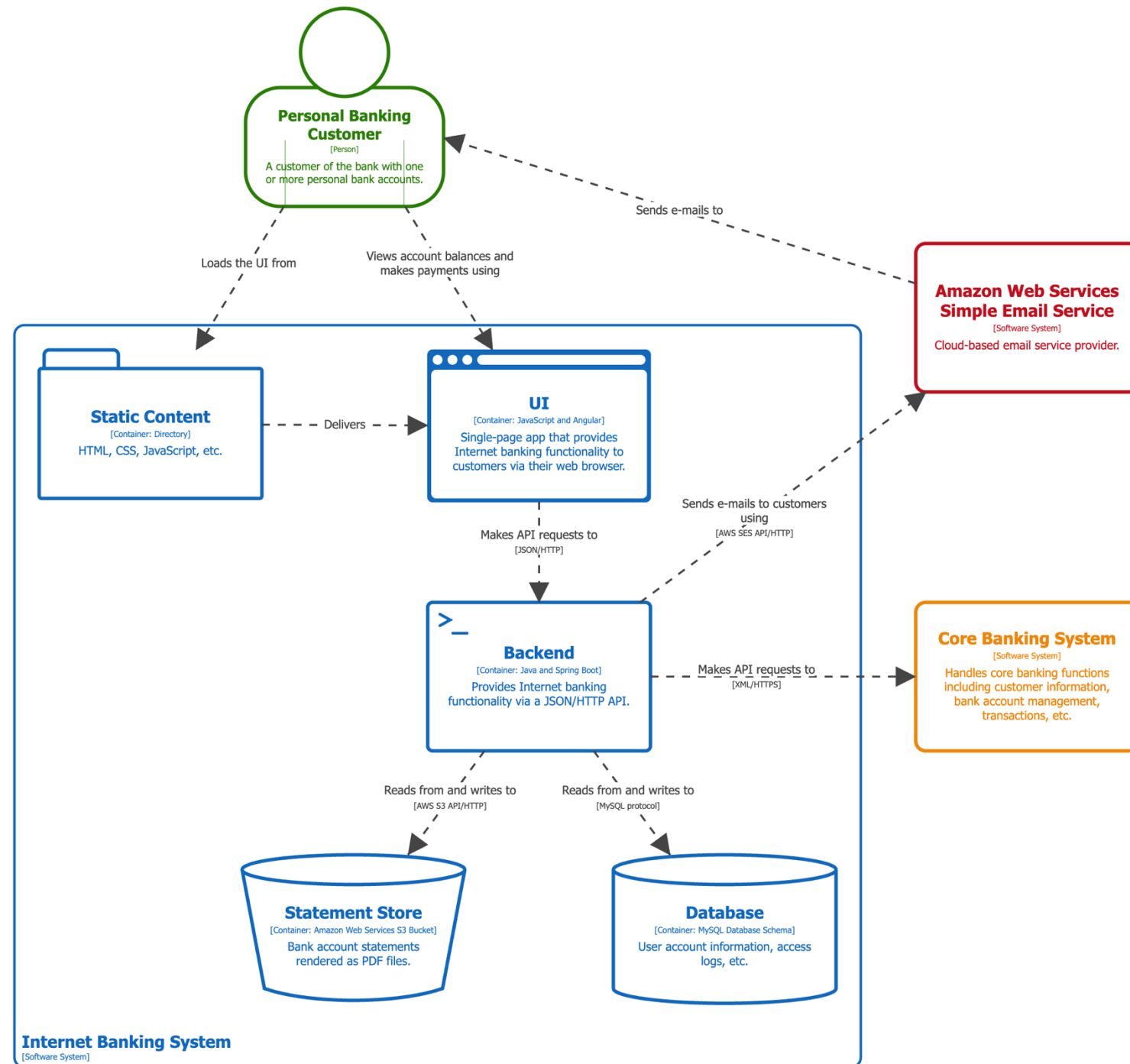
Arrows

labelled with what happens

**Big picture: no technology yet.
Readable by non-technical people.**

Source: Simon Brown, c4model.com · CC BY 4.0

C4: container diagram



Container View: Internet Banking System
The container diagram for the Internet Banking System | Simon Brown | c4model.com | License: CC BY 4.0

LEVEL 2 · CONTAINERS

KEY COMPONENTS

Container

an application or data store that runs on its own

Web app, single-page app, mobile app

what customers use

API application

server-side logic

Database

stores the data

Labels

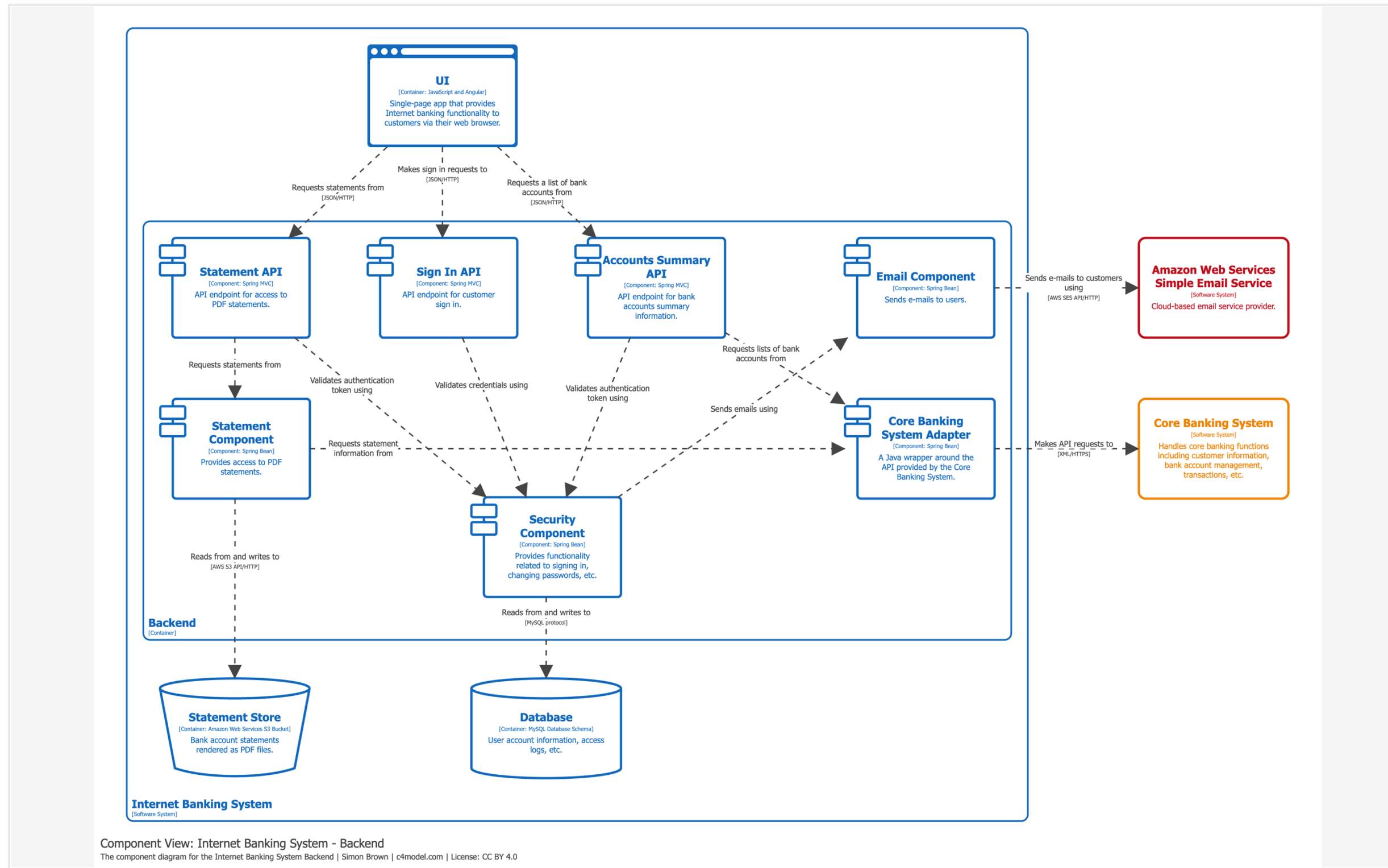
each box names its technology; each arrow its protocol

Zoom in: the main parts, their technology, and how they talk.

Source: Simon Brown, c4model.com · CC BY 4.0



C4: component diagram



LEVEL 3 · COMPONENTS

KEY COMPONENTS

Container in scope

the API application, drawn as a boundary

Components

its main parts: controllers, security, facades

Responsibilities

each box says what it does and its technology

Arrows

calls between components and to containers

Zoom in again: inside one container.
Optional.

Source: Simon Brown, c4model.com · CC BY 4.0

UML in brief

KEY FACTS

Booch, Rumbaugh, Jacobson; OMG standard since 1997

Fixed notation: every shape has a defined meaning

14 diagram types: structure and behavior

In industry: a few diagrams, used informally

POPULAR TOOLS

Diagrams as code

PlantUML

Mermaid

Drawing tools

draw.io

Lucidchart

Visio

o

Modeling tools

Enterprise Architect

Visual Paradigm

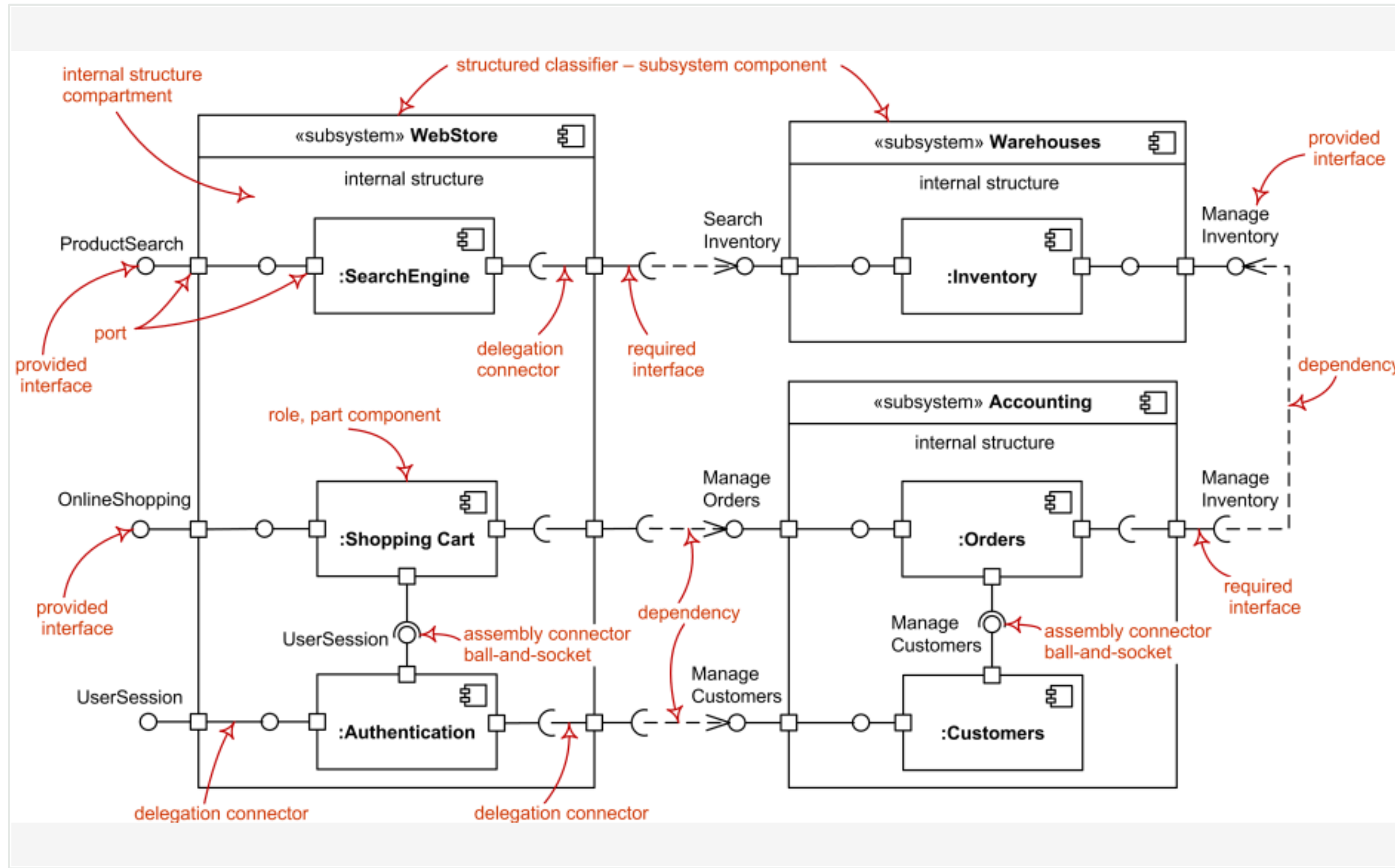
StarUML

UML standardizes the notation. C4 standardizes the abstractions.

Source: omg.org/spec/UML



UML: component diagram



UML 2

KEY COMPONENTS

Subsystem / part

«subsystem» WebStore contains parts like :SearchEngine

Provided interface (ball)

a service offered, e.g. ProductSearch

Required interface (socket)

a service needed, e.g. SearchInventory

Port

small square where interfaces attach

Delegation / assembly connector

port → inner part / ball meets socket

Dependency (dashed arrow)

one subsystem uses another's interface

WebStore uses Warehouses and Accounting only through their interfaces.

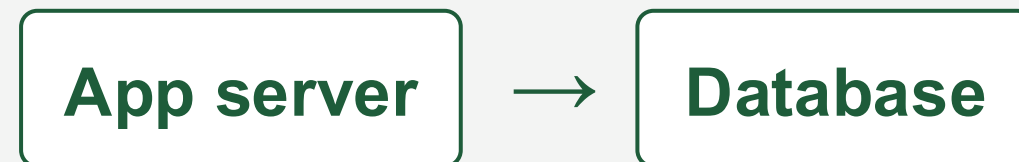
Source: K. Fakhroutdinov, uml-diagrams.org — “UML Component Diagrams Reference”

Quality attributes drive the structure

Feature: “Find documents by a word.”

TWO ARCHITECTURES

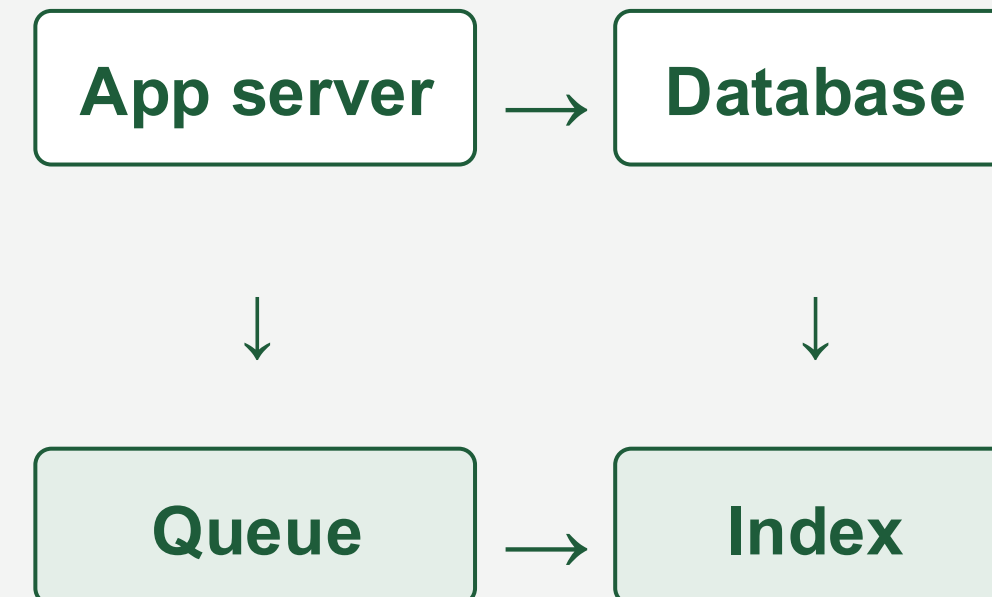
A few hundred documents



→ one query on the database

scale: small

Millions of documents, results in < 500 ms



→ full-text index, background indexing

performance: < 500 ms at millions of docs

Two laws

1 TRADE-OFFS

Everything in software architecture is a trade-off.

Cache: faster reads ↔ one more component to keep in sync

Separate processes: a crash stays contained ↔ more memory

Search index: fast search ↔ results lag behind updates

2 WHY?

Why is more important than how.

The code shows how, not why.

You can see the queue, but not that it keeps search under 500 ms.

Without the why, nobody can safely change it.

Someone removes the “unnecessary” queue and search slows down.

The why is what you re-check when things change.

If the reason no longer holds, the design can change.

— Richards & Ford, *Fundamentals of Software Architecture*

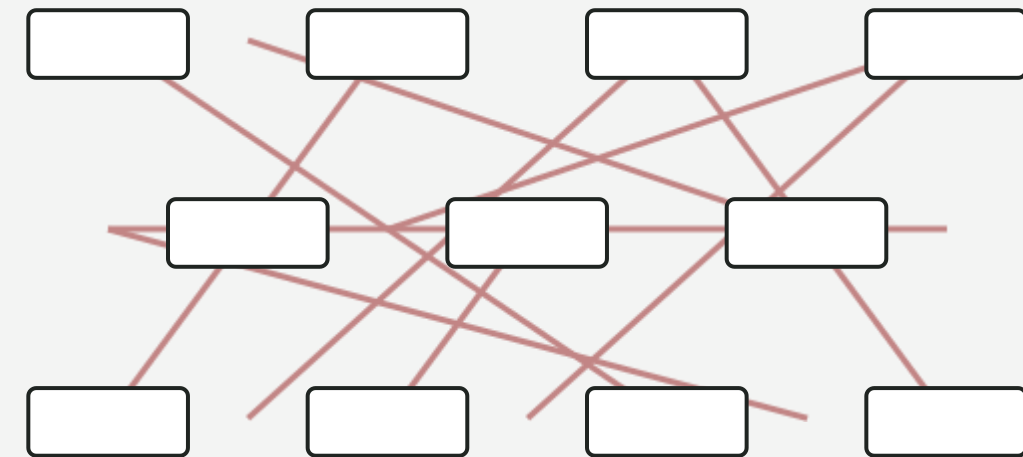


Good design goes unnoticed



✓ 0 incidents

Good design: nothing happens. Changes are easy. Looks obvious afterwards.



X pager: incident #14

Bad design: impressive, many parts, clever tricks — and frequent incidents.

“Good design is self-effacing: bad design is often more impressive than good.”

— S. Goedecke, 2025

Design principles

Ideas that make change cheaper

Modular design

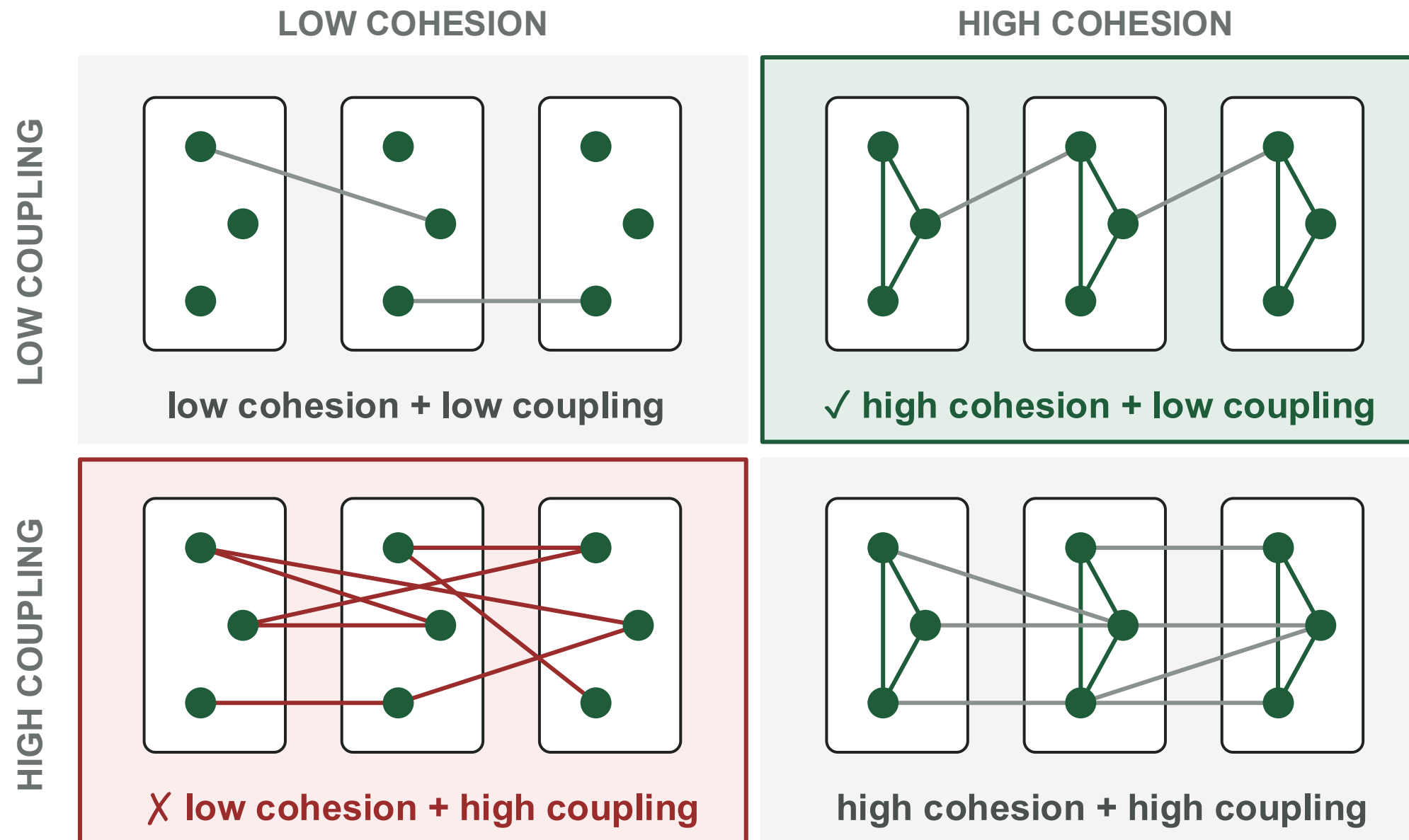
WITH GOOD MODULES

- ✓ Understand one part at a time
- ✓ Test parts on their own
- ✓ Reuse parts elsewhere
- ✓ Replace a part easily
- ✓ Fix and extend at lower cost

WITH TIGHT COUPLING

- ✗ A change ripples into other modules
- ✗ Can't test a module without its dependencies
- ✗ Can't reuse a module on its own
- ✗ Putting modules together takes more effort

Coupling and cohesion



Coupling: how much modules depend on each other → **keep low**

Cohesion: how much a module's parts belong together → **keep high**

Defined at function and class level; the same ideas apply to packages, services and systems.

Gitea team: a small feature meant changing many files. Why?

ENABLES changing, testing and reusing one module on its own

PREVENTS ripple effects and hidden dependencies between modules

Types of coupling

Worst (top): high coupling

Best (bottom): low coupling

| | | |
|---|---------|--|
| 1 | content | One module reaches into another's internal data or code |
| 2 | common | Modules share global data |
| 3 | control | One module passes a flag that steers another's logic |
| 4 | stamp | A whole record is passed when only a few fields are used |
| 5 | data | Only the needed data is passed, as parameters |



Content coupling (worst)

COUPLING worst **content** → common → control → stamp → data best

Content coupling: one module reaches into another's internals.

BEFORE

```
# customers.py (module A)
class CustomerList:
    def __init__(self):
        self._items = []

# lookup.py (module B)
def find_or_add(customers, name):
    for c in customers._items: # reads internals
        if c.name == name:
            return c
    c = Customer(name)
    customers._items.append(c) # edits A's data
    return c
```

FIX

```
# customers.py (module A)
class CustomerList:
    def __init__(self):
        self._items = []
    def find(self, name): ...
    def add_customer(self, name): ...

# lookup.py (module B)
def find_or_add(customers, name):
    c = customers.find(name)
    if c is None:
        c = customers.add_customer(name)
    return c
```

CONSEQUENCE CustomerList can't change how it stores customers without breaking lookup.py.

Control coupling

COUPLING worst **content** → **common** → **control** → **stamp** → **data** best

Control coupling: a flag tells another module which path to take.

BEFORE

```
def handle(order, action):  
    if action == "ship":  
        ... # pick, pack, label, notify  
    elif action == "refund":  
        ... # payment, stock, email  
    elif action == "cancel":  
        ... # release stock, close order  
  
# caller  
handle(order, action="refund")
```

FIX

```
def ship(order): ...  
def refund(order): ...  
def cancel(order): ...  
  
# caller  
refund(order)
```

CONSEQUENCE Three unrelated jobs behind one flag: each new action changes handle(), and callers must know its modes.

Reducing coupling doesn't mean zero connections. It means depending on less, and on more stable things.

Data coupling (best)

COUPLING worst **content** → **common** → **control** → **stamp** → **data** best

Data coupling: only the needed data is passed. (Before: stamp coupling.)

BEFORE (STAMP)

```
def late_fee(loan):  
    days = (date.today() - loan.due_date).days  
    return max(days, 0) * 0.25
```

```
# test: build a whole Loan  
# (book, member, dates...)
```

FIX (DATA)

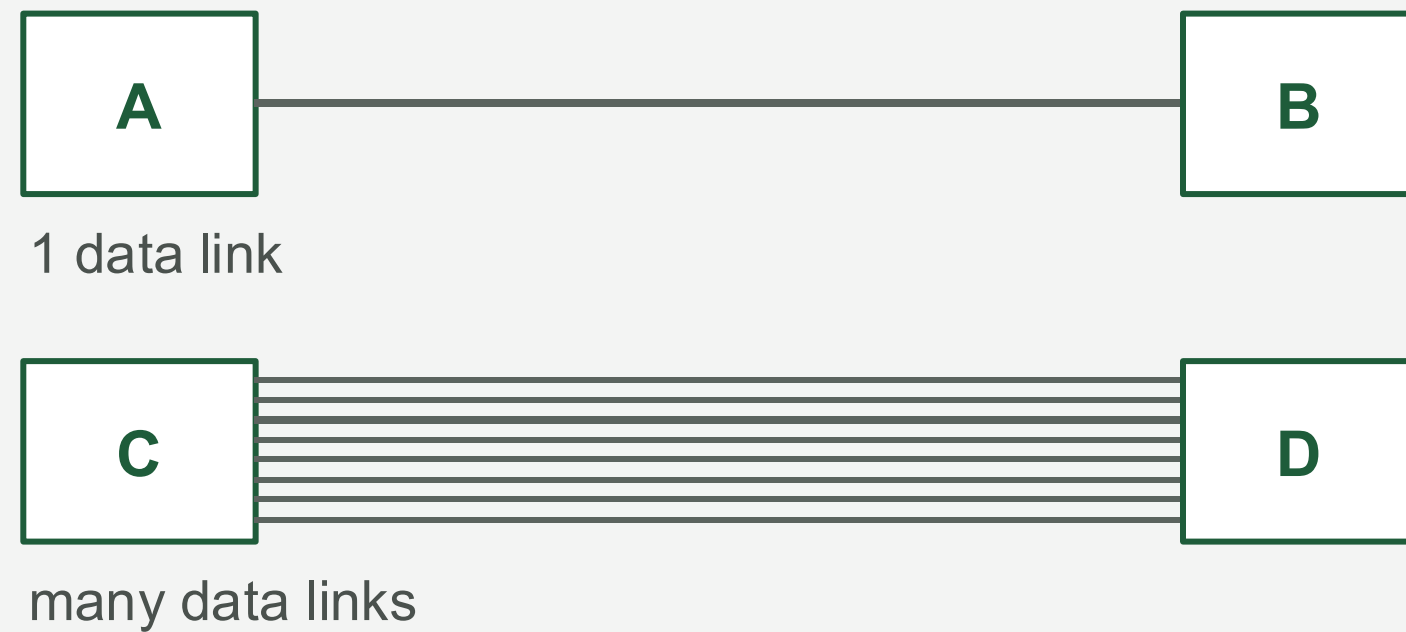
```
def late_fee(due_date, today):  
    days = (today - due_date).days  
    return max(days, 0) * 0.25  
  
late_fee(loan.due_date, date.today())  
  
# test: two plain dates  
late_fee(date(2026,10,1), date(2026,10,5))
```

CONSEQUENCE late_fee needs two dates, not a whole Loan: testable with plain values, reusable for anything with a due date.

Passing the whole object is fine when the function uses most of it.

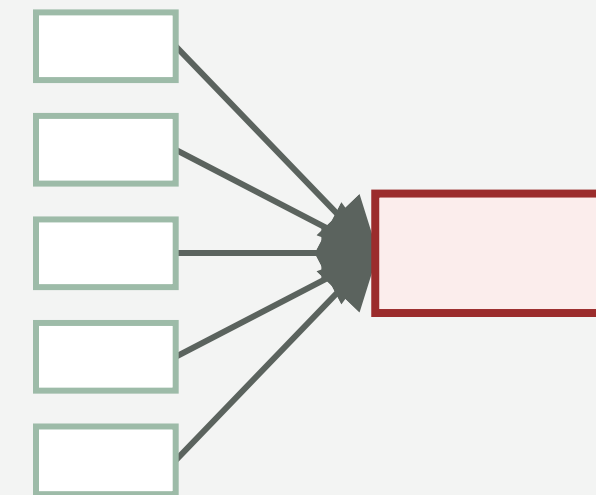
How much coupling?

KIND × AMOUNT



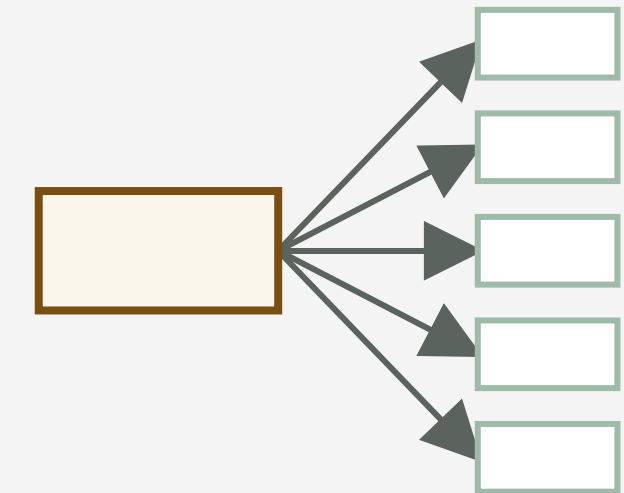
Many weak links still bind modules.

FAN-IN AND FAN-OUT



Fan-in

many depend on it → **risky to change**



Fan-out

depends on many → **fragile**

Coupling = kind × amount.

Types of cohesion

Worst (top): low cohesion

Best (bottom): high cohesion

| | | |
|---|-----------------|---|
| 1 | coincidental | Parts are unrelated; grouped by chance |
| 2 | logical | Parts are logically related but do different things |
| 3 | temporal | Parts run at the same time (e.g., startup) |
| 4 | procedural | Parts are related only by execution order |
| 5 | communicational | Parts work on the same data |
| 6 | informational | Separate operations on one data structure |
| 7 | functional | Every part contributes to one task |



Coincidental cohesion (worst)

COHESION worst **coincidental** → logical → temporal → procedural → communicational → informational → functional best

Coincidental cohesion: the parts have nothing in common.

BEFORE

```
# joes_stuff.py
def win2lin(path):
    ...

def days_since_epoch():
    ...

def output_report(data, stream):
    ...
```

FIX

```
# paths.py
def win2lin(path): ...

# dates.py
def days_since_epoch(): ...

# reports.py
def output_report(data, stream): ...
```

CONSEQUENCE Hard to find, hard to reuse: importing one function drags in the rest.

Logical cohesion

COHESION worst **coincidental** → **logical** → **temporal** → **procedural** → **communicational** → **informational** → **functional** best

Logical cohesion: parts are logically related but do different things (here, chosen by a flag).

BEFORE

```
# output.py
def output(kind, data, stream):
    if kind == "finance":
        ... # format report
    elif kind == "weather":
        ... # format forecast
    elif kind == "number":
        ... # format number
```

FIX

```
# finance_report.py
def write(data, stream): ...

# weather.py
def write(forecast, stream): ...

# numbers.py
def write(n, stream): ...
```

CONSEQUENCE Unrelated changes pile up in one function, and the kind flag is control coupling.

Functional cohesion (best)

COHESION worst **coincidental** → **logical** → **temporal** → **procedural** → **communicational** → **informational** → **functional** best

Functional cohesion: every part of the module contributes to one task.

EXAMPLE

```
# shipping.py
RATES = {"local": 4.0, "national": 7.5}

def shipping_cost(order):
    weight = total_weight(order.items)
    zone = zone_for(order.address)
    return RATES[zone] * weight

def total_weight(items): ...
def zone_for(address): ...
```

WHY IT'S GOOD

- ✓ One purpose: compute the shipping cost
- ✓ Test it: give an order, check a number
- ✓ Change rates or zones in one place
- ✓ Reuse or replace it as a unit

What is an interface?

An **interface** is what a module exposes to others: its **signatures** (names, inputs, outputs) and its **contract** (what it requires and what it promises).

Signature : names, parameter and return types, errors. A type checker can verify it.

Contract : preconditions (caller ensures), postconditions (module guarantees).

FUNCTION

shipping_cost(order)
-> float

order has items;
returns ≥ 0

CLASS

CustomerList
.add() .find()

find() returns None
if absent

MODULE

payments
charge() refund()

refund $\leq$ amount
charged

SERVICE

POST /bookings

same key $\rightarrow$
no second booking

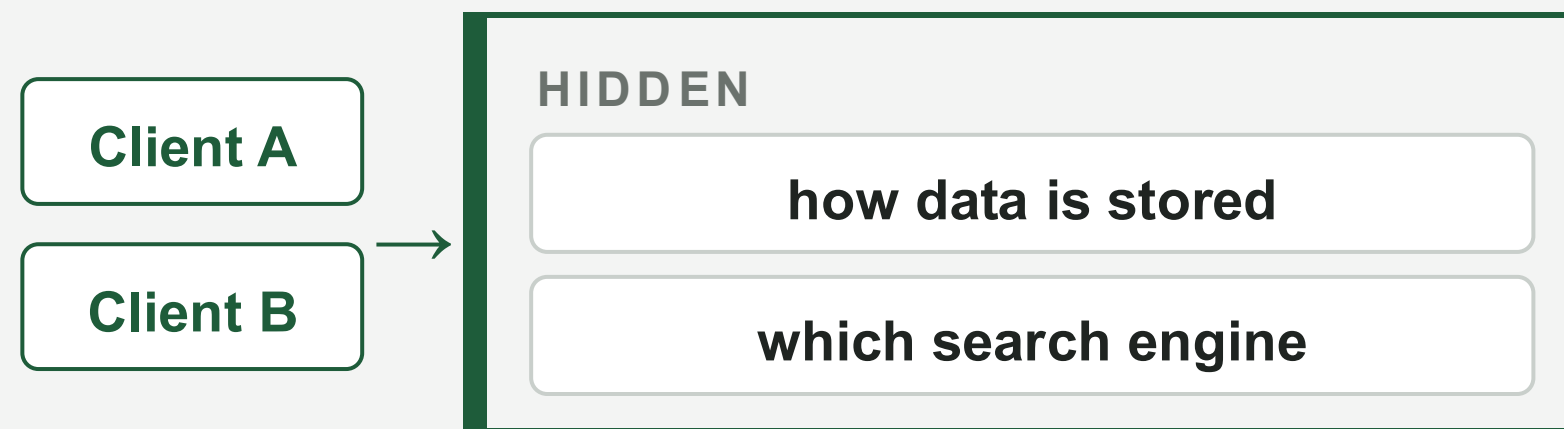
Stable interface: its signature and contract rarely change, even when the implementation does.

Information hiding

CONCEPT

Hide each **design decision likely to change** inside one module, behind a stable interface.

Encapsulation is the mechanism: keep data and code together and expose only the interface (in Python, by convention: `_items`).



— D. Parnas, 1972

EXAMPLE

```
# documents.py
class DocumentStore:
    def save(self, doc): ...
    def search(self, word): ...
    # hidden: SQLite or Postgres,
    #       query or search index

# caller
store.search("invoice")
```

Swap the database or add an index: callers don't change.

ENABLES changing a hidden decision without touching clients

PREVENTS content coupling: clients depending on internals

Deep interfaces

CONCEPT

A good interface is **small** and hides a lot.



✓ Deep

✗ Shallow

— J. Ousterhout, *A Philosophy of Software Design*

EXAMPLE

SHALLOW: CALLERS RUN THE STEPS

```
r = FileReader(path)
r.open_descriptor()
r.set_buffer_size(4096)
data = r.decode_bytes(r.read_chunk())
r.close_descriptor()
```

DEEP: ONE CALL

```
data = open(path).read()
```

Fewer things to depend on, fewer things to break.

Related: **orthogonality** (unrelated things change independently) · **interface segregation** (small, focused interfaces)

ENABLES using a module without learning its internals

PREVENTS stamp coupling: clients tied to more than they use

Depend on interfaces

Dependency inversion

The core depends on an **interface**, not on Stripe. Stripe depends on that interface.

Open/closed

Open to new providers, **closed** to edits: adding one doesn't change the core.



Medusa: payment → Stripe · file → local S3 · auth → email GitHub Google

Enables: adding a provider without editing the core

Prevents: lock-in to one vendor

Cost: an extra layer. Worth it only if you expect more providers

Medusa: payment providers

WHAT IT IS

Payment provider: an adapter between Medusa and one payment company.

Every provider implements:

initiate

authorize

capture

refund

cancel

Other providers:

file: local disk or S3

auth: email, GitHub or Google login

MEDUSA CODE (TYPESCRIPT, SIMPLIFIED)

```
// core: the interface
abstract class AbstractPaymentProvider {
  abstract authorizePayment(input)
  abstract capturePayment(input)
  abstract refundPayment(input)
}

// plugs in
class StripeProviderService
  extends AbstractPaymentProvider {
  capturePayment(input) { /* Stripe API */ }
}
```

```
// medusa-config.ts: chosen by config
providers: [{ resolve: ".../payment-stripe" }]
```

Checkout calls `capturePayment()`. The config decides who handles it.

DRY: Don't Repeat Yourself

DRY = Don't Repeat Yourself. Every piece of knowledge has **one authoritative place** in the system: knowledge, not code that merely looks alike. — Hunt & Thomas

COHESIVE, NOT DRY

OrderService

placeOrder()

TAX_RATE = 0.07

InvoiceService

issueInvoice()

TAX_RATE = 0.07

Rate changes: you must remember both.

DRY, NOT COHESIVE

utils

taxRate()

formatDate()

emailTemplate()

One copy, but the parts don't belong together.

BOTH

TaxRules

rate()

taxFor(order)

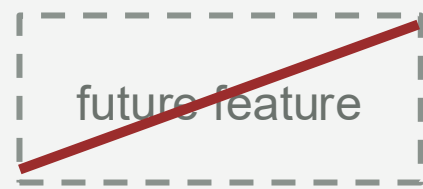
used by OrderService and InvoiceService

One place, and it belongs together.

PREVENTS copies that drift apart — the bug fixed in one copy but not the other

Build less

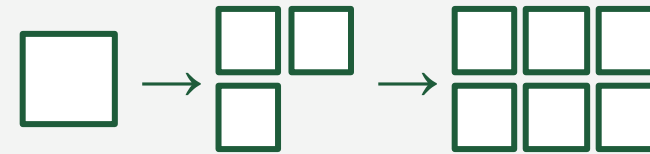
“A complex system that works is invariably found to have evolved from a simple system that worked.” — J. Gall



YAGNI: You Aren't Gonna Need It

Don't build it until you need it.

The “future” plug-in system nobody ever uses.



Start simple

Get a simple version working, then grow it.

Rate limiter: count in memory before adding Redis.

smoke detector

vs.

no flame

Remove, don't just handle

Change the design so the problem can't happen.

Report takes 20 s → compute it nightly, serve the file.

PREVENTS unused code that still needs maintenance, and complexity nobody needed

SOLID at a glance

S

Single responsibility

one reason to change

SAME IDEA AS

high cohesion

O

Open/closed

add behavior without editing the core

SAME IDEA AS

depend on interfaces

L

Liskov substitution

any implementation can replace another without breaking the code that uses it

SAME IDEA AS

safe to swap

I

Interface segregation

small, focused interfaces

SAME IDEA AS

deep interfaces

D

Dependency inversion

depend on abstractions, not details

SAME IDEA AS

depend on interfaces

Principle or violation?

A **Gitea:** issue search goes through one `Indexer` interface. Bleve, database, Elasticsearch or Meilisearch — chosen in config. The rest of Gitea never knows which indexer is used.

B **Cal.diy:** the function that builds the key that detects duplicate bookings exists in two files.

C **Cal.diy:** Google, Apple, CalDAV, Exchange... each implements the same `Calendar` interface (create events, read busy times, etc.). Booking code calls only the interface.

D **Gitea:** 65 of 78 files in `routers/web/repo` (web page handlers) import `models` (database code) directly, skipping `services` (business logic).

Which principle does each follow or break?

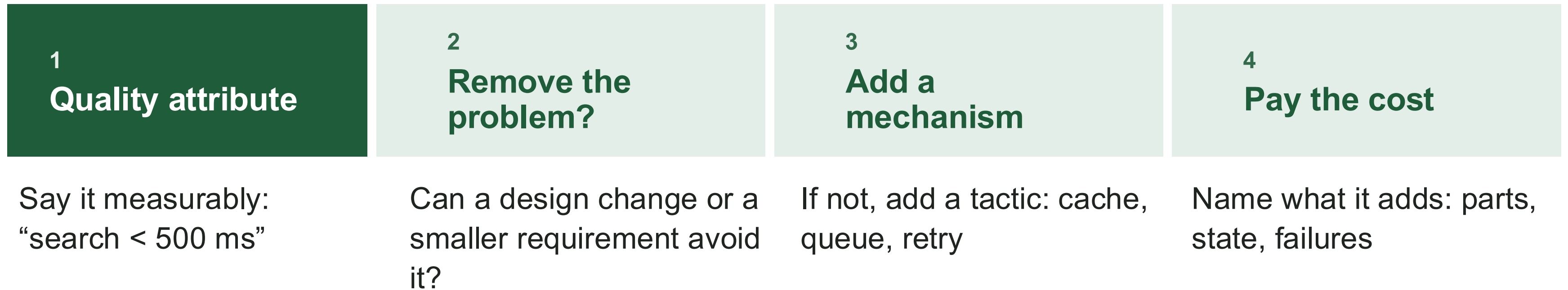
Principle or violation? — answers

| | VERDICT | WHY |
|---|-------------|---|
| A | ✓ follows | information hiding + depend on interfaces — the search engine is a hidden, swappable decision |
| B | ✗ breaks | one place for each fact (DRY) — if one copy changes, duplicate detection silently breaks |
| C | ✓ follows | depend on interfaces (open/closed) — add a calendar without editing the core |
| D | ~ trade-off | coupling — skipping the service layer: simpler handlers, but routers now depend on the data layer |

From quality attributes to design decisions

For each NFR: which design moves exist?

A design workflow



WHAT WE VISIT NEXT



Slow? Try these, in order

Performance & scalability

Remove the problem?

Add a mechanism

Pay the cost

MOVE

EXAMPLE REQUIREMENT → WHAT TO DO

1

Measure

not a move: find where

find the hot path

“Checkout < 1 s” → **one query takes 80%**

2

Index

let the DB do the work

“Orders by email < 200 ms” → **index on email**

3

Off the request

background jobs

“Confirm order < 500 ms” → **email + PDF in background**

4

Paginate

never load everything

“History stays fast as it grows” → **50 per page**

5

Scale out

stateless servers, replicas

“10× traffic on Black Friday” → **more servers + replica**

6

Cache last

adds state: stale data

“Product page < 200 ms, prices 1 min old OK” → **cache 60 s**

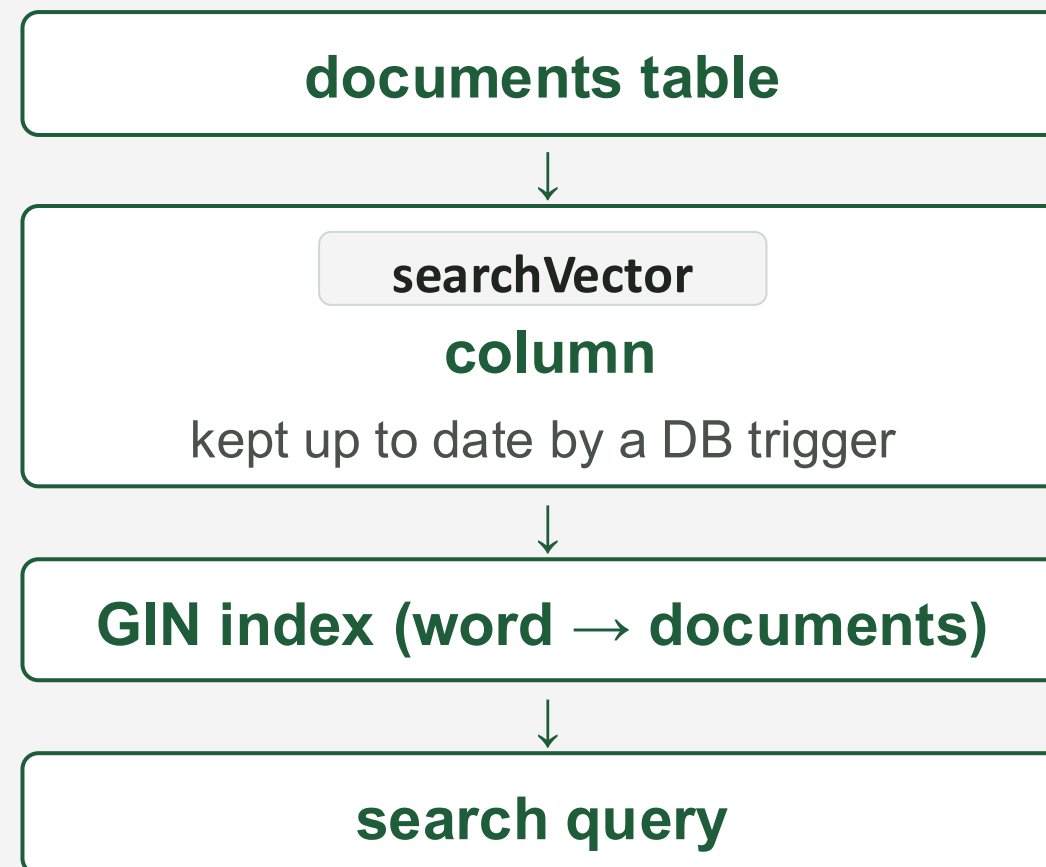


Two projects, two moves

Move 2 · Index

Outline

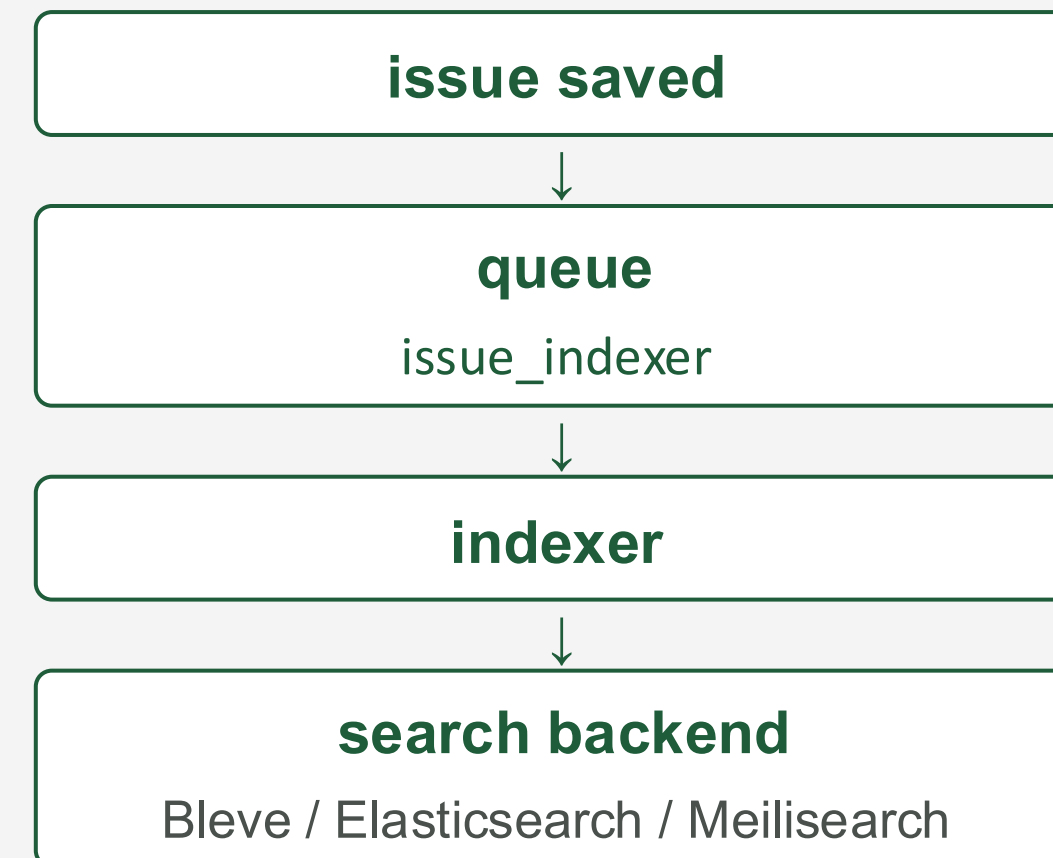
Search looks up words in an index instead of scanning every document.



Move 3 · Off the request

Gitea

Saving an issue returns right away. Indexing happens later, in the background.

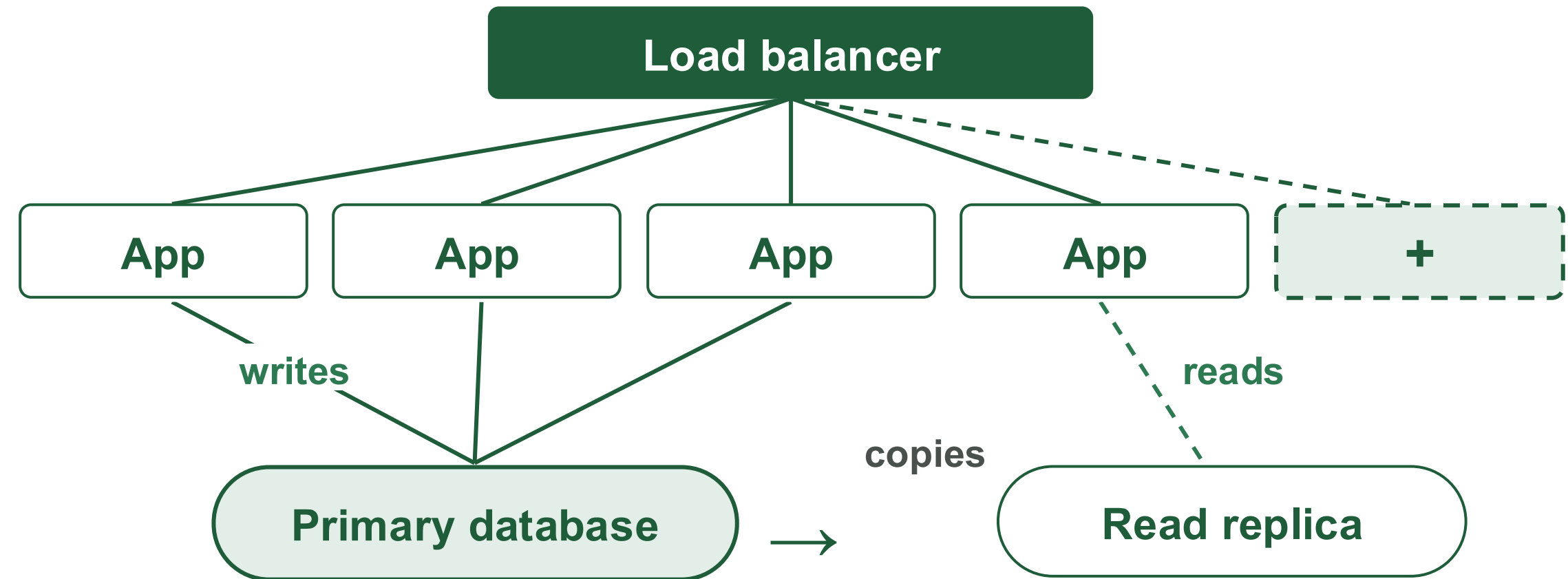


~ Gitea's code search is **off by default** : it uses a lot of disk (trade-off)

Stateless servers scale out

Stateless app servers → add more as load grows

State in **one place** → the database (+ replicas)



State

logins, carts, uploads → database, Redis, S3

Why it scales

any server can handle any request

Cost

replicas lag behind; send reads that must be fresh to the primary

Tools: Performance & scalability

Measure

browser DevTools

EXPLAIN ANALYZE

Datadog

Sentry

Load balancing

Nginx

HAProxy

AWS Application Load Balancer

Scale out

Docker

Kubernetes

Google Cloud Run

Cache and background jobs

Redis

Cloudflare CDN

Celery

Sidekiq

Examples, not recommendations.



Things fail? From basic to costly

Reliability

Remove the problem?

Add a mechanism

Pay the cost

MOVE

EXAMPLE REQUIREMENT → WHAT TO DO

1

Redundancy

replicas take over

“Survive a server failure” → **primary + replica**

2

Backups

restore after mistakes

“Recover from a bad delete” → **nightly backup, tested restore**

3

Idempotent operations

a repeat has no extra effect

“Double click books once” → **unique key**

4

Retries with backoff

wait 1 s, 2 s, 4 s...

“Ride out a network blip” → **retry 3 times**

5

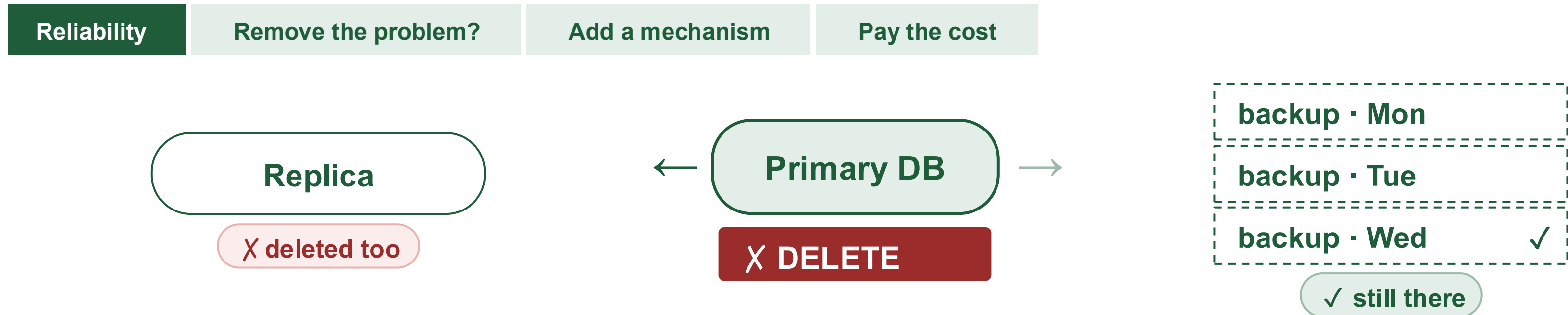
Circuit breaker

adds state, new ways to fail

“Store still works if the reviews service is down” → **after 5 failures, skip reviews for 30 s**



Redundancy: replicas and backups



Replica: a live copy

Updated: continuously, within ms

Protects against: a machine failure (takes over)

Doesn't protect: mistakes (it copies them)

Backup: a copy from the past

Updated: on a schedule, e.g. nightly

Protects against: bugs, bad deletes, ransomware

Doesn't protect: changes since the last backup

Test your backups: restore them regularly.

GitLab.com, January 31, 2017

~300
GB

deleted from the
primary database

An engineer deletes the database directory on the **primary** , not the replica

4 ways to recover the data. 3 failed. The 4th was luck.

1



Replication

had stopped

2



**pg_dump
backups**

wrong version →
empty; alert
emails rejected

3



**Disk
snapshots**

never enabled

4



LVM snapshot

on staging, 6 h
old: luck

~6 hours of data lost · ~18 hours to recover



What happens when they click twice?

Cal.diy: a booker clicks **Confirm** twice — or the network retries.

How would you make sure the host isn't double-booked?

Bonus: remove the problem or handle it?

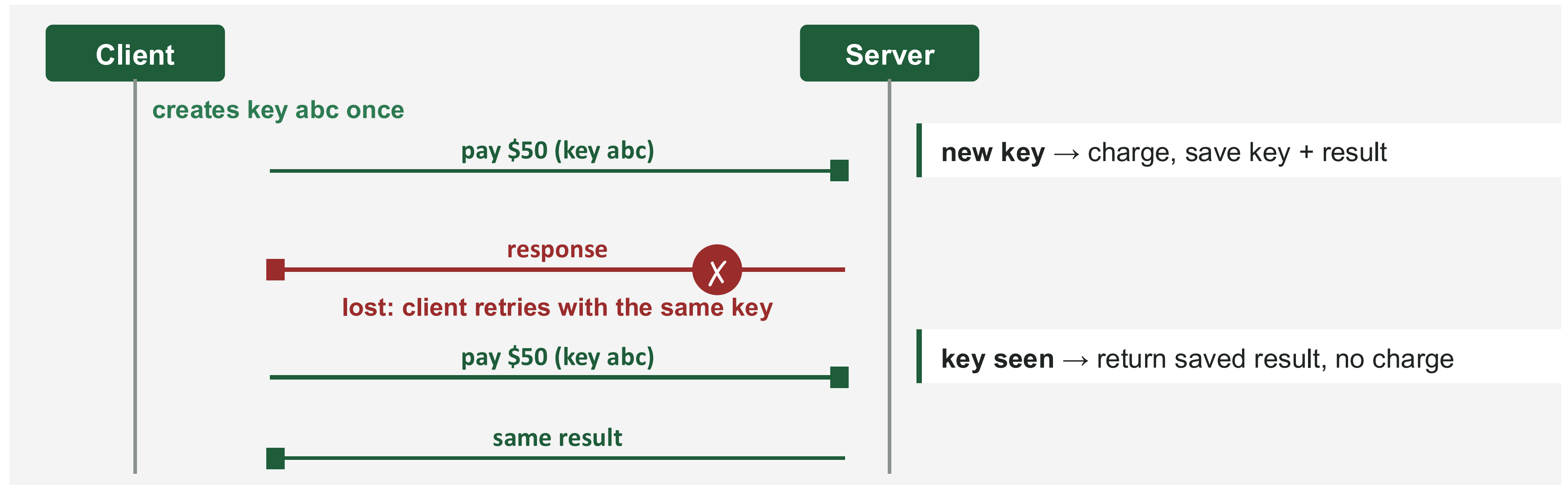
BOOKING
30 min meeting
Tue, Oct 6 · 10:00 – 10:30

| | | |
|------|-------|-------|
| 9:30 | 10:00 | 10:30 |
|------|-------|-------|

Confirm ×2

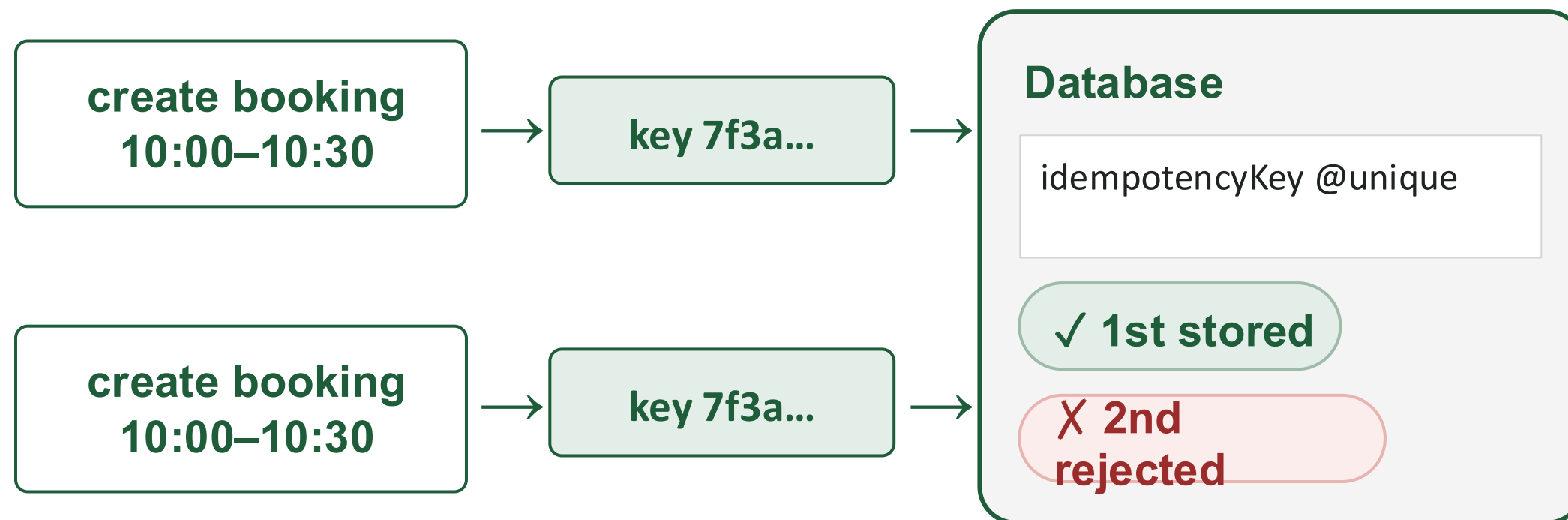
Retries need idempotency

A request times out. Did it happen? **Retry** safely, with an **idempotency key**.



Prevents: double charges, duplicate orders and bookings, and data that no longer adds up (two payments for one order).

What happens when they click twice? — answer



- 1 **Key = hash(start, end, host)** : computed from the slot, not random, so same slot → same key
- 2 The database requires a **unique** key: the 2nd insert is rejected
- 3 Stops double clicks, retries, and two bookers alike
- 4 Cancel or reject → key cleared → slot free again

Overlapping slots (10:00 vs. 10:15) get different keys; an availability check handles those.

No race, nothing to forget: the problem is removed, not handled.

When a dependency fails

Reliability

Remove the problem?

Add a mechanism

Pay the cost

Circuit breaker

The product page calls a **reviews service**. It goes down.

X Without: every page waits for a timeout; checkout slows too

- 1 After 5 failures, the breaker opens
- 2 For 30 s, skip reviews: “Reviews unavailable”
- 3 Then try once; if it works, close again

Retries: wait 1 s, 2 s, 4 s... so you don't pile on.

Fail open or closed?

A **check** itself breaks. Let the request through, or block it?

Ask: which is worse, wrongly letting in or wrongly keeping out?

open let it through

Rate limiter down: blocking all would take the site down

closed block it

Login check down: a short lockout beats exposing data

Decide when designing, not by accident.

Tools: Reliability

Database replicas

Postgres replication

MySQL replication

AWS RDS

Cloud SQL

Database backups

pg_dump

pgBackRest

managed snapshots

Files

S3

restic

versioning

Retries, idempotency, circuit breakers

idempotency keys (Stripe API)

unique constraints

Resilience4j

Poll
y

Examples, not recommendations.

